

## 2/9/26 DAY 10:

---

### Member vs Non-member Operators

#### Member Operator Overloading

- It is done by defining a member function in the class.
- The left-hand operand must be an object of the class.
- The right-hand operand can be of any type.

Syntax:

```
ClassName operator OperatorSymbol (const ClassName &obj) {  
    // Implementation  
}
```

Example:

```
Complex operator+(const Complex &other) {  
    return Complex(real + other.real, imag + other.imag);  
}
```

```
Complex c1(3, 4);  
Complex c2(1, 2);  
Complex c3 = c1 + c2; // Uses the overloaded + operator
```

#### Non-member Operator Overloading

- It is done by defining a non-member function.
- The left-hand operand can be of any type.
- The right-hand operand must be an object of the class.

Syntax:

```
ClassName operator OperatorSymbol (const ClassName &obj1, const ClassName &obj2) {  
    // Implementation  
}
```

Example:

```
Number operator+(const int num1, const Number &num2) {  
    return Number(num1 + num2.value);  
}
```

This allows for expressions where the left-hand operand is not an object of the class, enabling more flexible usage of the operator.

```
Number n1(5);  
  
Number n3 = n1 + 10; // it can use the member overloaded +  
Number n4 = 10 + n1; // it can use the non-member overloaded +
```

## Subscript Operator [ ]

- The subscript operator can be overloaded to provide array-like access to class objects.
- The overloaded subscript operator can be defined as a member function or a non-member function.
- It can return a reference to the element at the specified index.
- It is commonly used in classes that represent collections or arrays.
- It is important to handle out-of-bounds access appropriately, either by throwing an exception or returning a default value.
- It is also possible to overload the subscript operator for both const and non-const objects, allowing read-only access for const objects.

Syntax:

```
ClassName& operator[](int index) {  
    // Implementation  
}
```

Example:

```
class Array {  
private:  
    int* arr;  
    int size;  
public:  
    Array(int s) : size(s) {  
        arr = new int[size];  
    }  
  
    int& operator[](int index) {  
        if (index < 0 || index >= size) {  
            throw std::out_of_range("Index out of bounds");  
        }  
        return arr[index];  
    }  
}
```

```

    }
};

int main() {
    Array myArray(5);
    myArray[0] = 10;
    myArray[1] = 20;
    std::cout << myArray[0] << std::endl; // Output: 10
    std::cout << myArray[1] << std::endl; // Output: 20
    return 0;
}

```

## Stream Operators << and >>

- The stream insertion (<<) and extraction (>>) operators can be overloaded to enable input and output operations for class objects.
- The overloaded stream operators are typically defined as non-member functions.
- The stream insertion operator (<<) is used for output, while the stream extraction operator (>>) is used for input.
- The overloaded stream operators should return the stream object to allow chaining of operations.

Syntax:

```

std::ostream& operator<<(std::ostream &out, const ClassName &obj) {
    // Implementation for output
    return out;
}

std::istream& operator>>(std::istream &in, ClassName &obj) {
    // Implementation for input
    return in;
}

```

Example:

```

class Complex {
private:
    int real;
    int imag;
public:
    Complex(int r = 0, int i = 0) : real(r), imag(i) {}

    friend std::ostream& operator<<(std::ostream &out, const Complex &c) {
        out << c.real << " + " << c.imag << "i";
        return out;
    }

    friend std::istream& operator>>(std::istream &in, Complex &c) {

```

```
        in >> c.real >> c.imag;
        return in;
    }
};
```

## Friend Functions

- A friend function is a function that is not a member of a class but has access to its private and protected members.
- It is declared using the **friend** keyword inside the class definition.
- Friend functions can be useful for operator overloading and for providing access to private data when necessary.

Syntax:

```
class ClassName {
    friend ReturnType FriendFunctionName(Parameters);
    // Other members of the class
};
```

Example:

```
class Box {
private:
    double width;
public:
    Box(double w) : width(w) {}
    friend void printWidth(Box &b);
};
```

## Friend Classes

- A friend class is a class that is granted access to the private and protected members of another class.
- It is declared using the **friend** keyword inside the class definition.
- Friend classes can be useful for closely related classes that need to access each other's private data.

Syntax:

```
class ClassName {
    friend class FriendClassName;
    // Other members of the class
};
```

Example:

```
class Box {  
private:  
    double width;  
public:  
    Box(double w) : width(w) {}  
    friend class BoxPrinter;  
};  
class BoxPrinter {  
public:  
    void printWidth(Box &b) {  
        std::cout << "Width of box: " << b.width << std::endl;  
    }  
};
```

## Abstraction Concept

- Abstraction is a fundamental concept in object-oriented programming that focuses on exposing only the essential features of an object while hiding the unnecessary details.
- It allows developers to create simplified models of complex systems, making it easier to understand and work with them.
- Abstraction can be achieved through the use of abstract classes and interfaces, which define a contract for derived classes to implement specific behaviors without revealing the underlying implementation details.

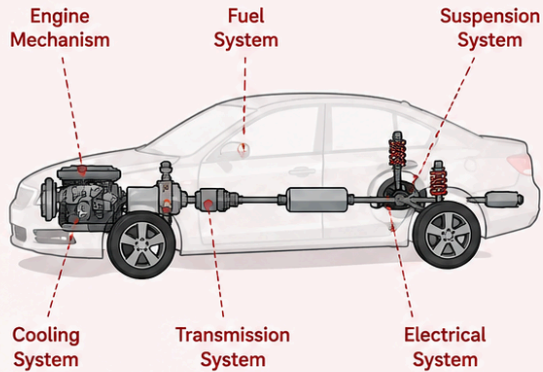
# ABSTRACTION

## Show Essential. Hide Unnecessary.



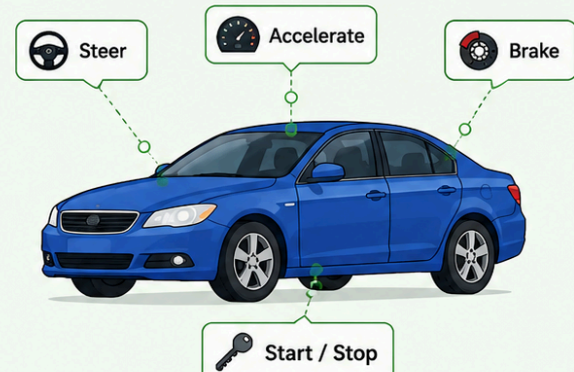
Abstraction is the concept of exposing only the **essential features** to the user and hiding the **internal implementation details**.

### WHAT'S INSIDE? (HIDDEN)



Complexity is hidden from the user.

### WHAT THE USER SEES (ABSTRACTION)



User interacts with simple, easy-to-use features.

### KEY TAKEAWAYS



Hide internal details.  
(Data + Implementation)



Expose only what is necessary.  
(Behavior / Interface)



Reduces complexity for the user.



Helps in building better, maintainable systems.



Focus on WHAT it does, not HOW it does.

### REAL-WORLD EXAMPLES



**ATM Machine**  
You don't need to know how it verifies your PIN internally.



**Smartphone**  
You just tap icons; you don't know how the OS manages memory.



**Washing Machine**  
You select a mode; you don't know how it controls water level, motor speed, etc.



**Car Driving**  
You drive using controls; you don't know the internal working of the engine.

## Association

- Association is a relationship between two classes that establishes a connection between their objects.
- It represents a "like-a" or "uses-a" or "needs-a" relationship, where one class can use the functionality of another class without owning it.
- Association can be one-to-one, one-to-many, or many-to-many, depending on the nature of the relationship between the classes.

# ASSOCIATION

*Objects Know or Interact with Each Other.*



Association is a relationship where one object is aware of another object and they interact with each other, but they are **independent**.

## TEACHER



### Attributes:

- id
- name
- subject

### Behaviors:

- teach()
- assignHomework()

Knows /  
Interacts with

Teacher knows  
Student  
Student knows  
Teacher

## STUDENT



### Attributes:

- id
- name
- grade

### Behaviors:

- study()
- submitAssignment()

## KEY POINTS



Objects are connected by a relationship.



They can live independently.



Changes in one object do not destroy the other.



Represents "uses", "knows", "interacts" relationship.



No ownership.  
No lifetime dependency.



Often implemented using references, pointers, or IDs.

## REAL-WORLD EXAMPLES

### Customer – Order



Customer places many orders.  
Order stores customer info.  
**Both exist independently.**

### Doctor – Patient



Doctor treats many patients.  
Patient can change doctor.  
**Both exist independently.**

### Library – Book



Library has many books.  
Book can exist in another library.  
**Both exist independently.**

### Department – Employee



Employee works in a department.  
Employee can move to another.  
**Both exist independently.**



Association models a connection, not a ownership. It helps objects communicate without being tied to each other's lifetime.

## Aggregation

- Aggregation is a specialized form of association that represents a "has-a" relationship between two classes.
- In aggregation, one class (the whole) contains or is composed of other classes (the parts), but the parts can exist independently of the whole.
- Aggregation is weaker than composition, as it allows the contained objects to exist independently of the container object.
- Aggregation is often used to model relationships where the lifetime of the contained objects is not dependent on the lifetime of the container object.

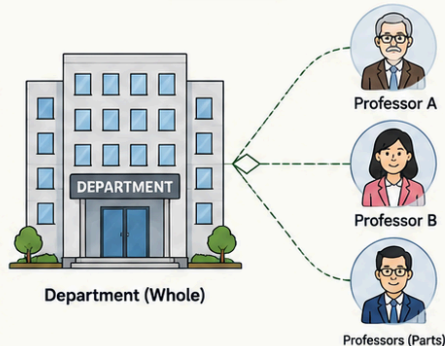
# AGGREGATION

## Whole-Part Relationship with Independent Lifetime



Aggregation is a special form of association where one object contains (has) another object as a part, but the part can exist **independently** of the whole.

### CONCEPT



A Department has Professors.  
Professors can exist even if the Department is deleted.

### UML REPRESENTATION



Hollow diamond (◇) at the whole end represents aggregation.

- 1 Department can have 0 to many Professors.
- Each Professor belongs to at most one Department at a time.
- Professors are independent.

### KEY POINTS



Represents a HAS-A relationship.



Parts can exist **independently**.



Deleting the whole does NOT delete the parts.



Usually implemented using references or pointers.



Aggregation ≠ Composition

### REAL-WORLD EXAMPLES

#### Team – Player



A team has many players.  
Players can play for other teams or exist without a team.

#### Library – Book



A library has many books.  
Books can exist without being in any particular library.

#### University – Professor



A university has many professors.  
Professors can work in other universities or independently.

#### Playlist – Song



A playlist has many songs.  
Songs can exist in many playlists or without any playlist.



**THINK IN SHORT:** Aggregation = I have you, but you can live without me. (Whole and part have an **independent lifetime**.)

## Composition

- Composition is a specialized form of aggregation that represents a "has-a" relationship between two classes.
- In composition, one class (the whole) contains or is composed of other classes (the parts), and the parts cannot exist independently of the whole.
- Composition is stronger than aggregation, as it implies a more intimate relationship between the classes.

# COMPOSITION

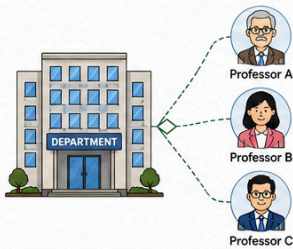
## Strong Whole–Part Relationship with Dependent Lifetime



Composition is a strong form of aggregation where one object **owns** another.  
The part **cannot** exist **independently** of the whole.

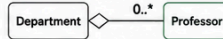
### AGGREGATION (for comparison)

Department has Professors



- ✓ Professors can exist even if Department is deleted.
- ✓ Department does **NOT** own Professors.

#### UML Notation



#### C++ Implementation (Example)

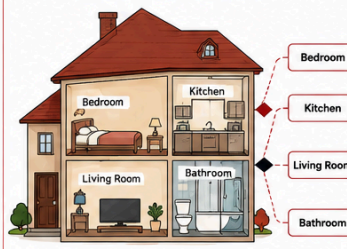
```
class Professor {
    string name;
};

class Department {
    // Non-owning relationship
    vector<Professor*> professors;
};
```

Uses references or pointers.  
DOES NOT delete the parts.

### COMPOSITION

House contains Rooms



- ✓ Rooms cannot exist without the House.
- ✓ Deleting the House deletes all the Rooms.  
**House OWNS** the Rooms.

#### UML Notation



#### C++ Implementation (Example)

```
class Room {
    string name;
public:
    Room(string n) : name(n) {}
};

class House {
    // Owning relationship
    vector<Room> rooms; // or
    // vector<unique_ptr<Room>> rooms;
};
```

Uses objects or smart pointers.  
**DELETES** the parts.



#### KEY DIFFERENCE

#### Aggregation

Part can live independently.  
(Weak ownership)



#### Composition

Part cannot live independently.  
(Strong ownership)



Think: Aggregation = I have you,  
Composition = I own you.

### REAL-WORLD EXAMPLES



#### College – Student

College has Students.  
Students can exist even  
if College is deleted.



#### Team – Player

Team has Players.  
Players can play for  
other teams.



#### House – Room

House contains Rooms.  
Rooms cannot exist  
without the House.



#### Car – Engine

Car contains Engine.  
Engine cannot exist  
without the Car.



**COMPOSITION** is a stronger relationship than **AGGREGATION**. | Choose the right relationship to model real-world scenarios correctly!

## Relationship Modeling

- Relationship modeling is the process of defining and representing the relationships between different classes in a system.
- It helps in understanding how classes interact with each other and how they are connected.
- Object-oriented design often involves creating class diagrams that illustrate the relationships between classes, including associations, aggregations, and compositions.

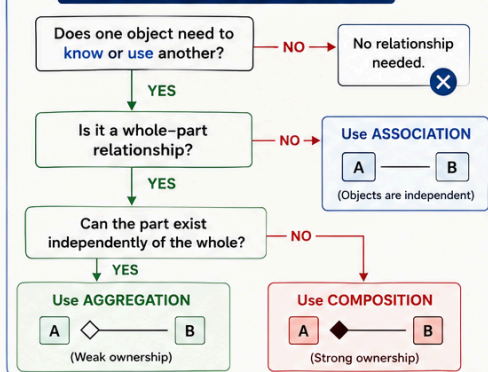
# RELATIONSHIP MODELING

Choose the Right Relationship for the Right Meaning



Not every "has-a" is the same! Choose based on **ownership** and **lifetime dependency**.

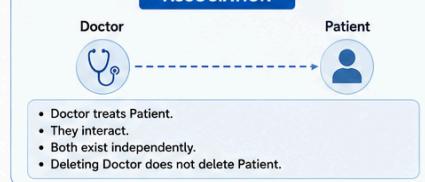
## HOW TO DECIDE? (QUICK GUIDE)



## RELATIONSHIP COMPARISON

| Aspect                    | ASSOCIATION                               | AGGREGATION                                         | COMPOSITION                                              |
|---------------------------|-------------------------------------------|-----------------------------------------------------|----------------------------------------------------------|
| Meaning                   | Objects know or interact with each other. | Whole has a part, but part can exist independently. | Whole owns the part, part cannot exist independently.    |
| UML Notation              | A — B                                     | A o-- B                                             | A *-- B                                                  |
| Ownership                 | No ownership                              | Weak ownership                                      | Strong ownership                                         |
| Lifetime Dependency       | Independent                               | Independent                                         | Dependent                                                |
| When Whole is Deleted     | No effect on the other                    | Part still exists                                   | Parts are destroyed                                      |
| Implementation (C++ hint) | Reference / Pointer<br>A* b; B* a;        | Reference / Pointer (non-owning)<br>A* b; B* a;     | Object / Smart Pointer (owning)<br>A b; unique_ptr<B> b; |

## ASSOCIATION

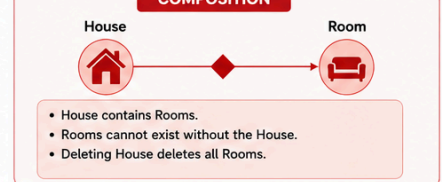


## REAL-WORLD EXAMPLES

### AGGREGATION



### COMPOSITION



## KEY TAKEAWAYS



Start with the **meaning**, not the code.



Think about **lifetime dependency**.



Composition means **responsibility**.



The right relationship leads to **maintainable designs**.



Model the real world **accurately** in code.



Good designs are not about using more relationships, but about using the **RIGHT** relationship for the **RIGHT** reason.

## Inheritance Concept

- Inheritance is a fundamental concept in object-oriented programming that allows a class (derived class) to inherit properties and behaviors (attributes and methods) from another class (base class).
- It promotes code reusability and establishes a hierarchical relationship between classes.

# INHERITANCE CONCEPT

Build New Classes from Existing Ones.



Inheritance is a mechanism where a new class (**derived class / child class**) acquires the properties and behaviors (data + functions) of an existing class (**base class / parent class**).

## Why Inheritance?

- ✓ Reusability of code
- ✓ Extensibility
- ✓ Maintainability
- ✓ Models "is-a" relationship

## THE BIG IDEA

### Real World



A Car is a Vehicle.  
Car has all features of Vehicle and some additional features.



### In OOP

Vehicle (Base / Parent)

is-a

General class  
(common properties and behaviors)

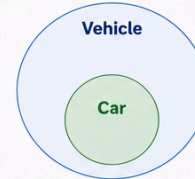
More specific class  
(inherits + adds new things)

Car (Derived / Child)

## IS-A RELATIONSHIP

It represents an "is-a" relationship.

Example:  
Car IS-A Vehicle



- Every **Car** is a **Vehicle**.  
(derived is a type of base)
- But not every **Vehicle** is a **Car**.  
(base is more general)

## HOW INHERITANCE WORKS

### Base Class (Parent)

Defines common data and functions.



Data Members



Member Functions

Base Class

inherit

Derived Class

### Derived Class (Child)

Inherits from base class and can:

- ✓ Use inherited members
- ✓ Add new members
- ✓ Override (redefine) some members

## EXAMPLE IN C++

```
// Base Class
class Vehicle {
protected:
    int wheels;
public:
    void start() {
        cout << "Vehicle started\n";
    }
};
```

```
// Derived Class
class Car : public Vehicle { // Car IS-A Vehicle
private:
    string brand;
public:
    void setBrand(string b) { brand = b; }
    void show() {
        cout << "Car Brand: " << brand
            << ", Wheels: " << wheels << "\n";
    }
};
```

## SYNTAX

```
class DerivedClass : accessSpecifier BaseClass {
// members
};
```

Derived (Child)

Mode of inheritance  
(public / protected / private)

Base (Parent)

## Access Specifiers in Inheritance

| Specifier | Meaning                                          |
|-----------|--------------------------------------------------|
| public    | Public members of base remain public in derived. |
| protected | Public & protected become protected in derived.  |
| private   | Public & protected become private in derived.    |

## KEY TAKEAWAYS



Promotes code reuse.



Supports extensibility.



Builds a natural hierarchy.



Follows "is-a" relationship.



Foundation of OOP design.



Inheritance helps us create new classes by reusing and extending existing ones, forming a hierarchy that models real-world relationships effectively.

## Real-World Analogy:

A Dog IS-A Animal.

Dog inherits all properties of Animal and adds its own.

IS-A

IS-A



## Base Class and Derived Class

- A base class (also known as a parent class or superclass) is a class that provides common attributes and methods that can be inherited by other classes.
- A derived class (also known as a child class or subclass) is a class that inherits from a base class and can extend or override its functionality.

Example:

```
class Base {
public:
    void display() {
        std::cout << "This is the base class." << std::endl;
    }
};

class Derived : public Base {
public:
    void show() {
        std::cout << "This is the derived class." << std::endl;
    }
};
```

## Modes of Inheritance

- There are different modes of inheritance in C++ that determine the accessibility of base class members in the derived class. The modes of inheritance are:
  1. **Public Inheritance:** The public and protected members of the base class become public and protected members of the derived class, respectively.
  2. **Protected Inheritance:** The public and protected members of the base class become protected members of the derived class.
  3. **Private Inheritance:** The public and protected members of the base class become private members of the derived class.

| Base Class Members | Public Inheritance | Protected Inheritance | Private Inheritance |
|--------------------|--------------------|-----------------------|---------------------|
| Public             | Public             | Protected             | Private             |
| Protected          | Protected          | Protected             | Private             |
| Private            | Private            | Private               | Private             |

## Types of Inheritance

- There are several types of inheritance in C++ that define how classes can inherit from one another. The main types of inheritance are:

### Single Inheritance

- In single inheritance, a derived class inherits from a single base class. This is the simplest form of inheritance and establishes a one-to-one relationship between the base and derived classes.

```
class Base {  
    // Base class members  
};  
class Derived : public Base {  
    // Derived class members  
};
```

### Multiple Inheritance

- In multiple inheritance, a derived class inherits from more than one base class. This allows the derived class to combine the features of multiple base classes.

```
class Base1 {  
    // Base1 class members  
};  
class Base2 {  
    // Base2 class members  
};  
class Derived : public Base1, public Base2 {
```

```
// Derived class members  
};
```

## Multilevel Inheritance

- In multilevel inheritance, a derived class inherits from another derived class, forming a chain of inheritance.

```
class Base {  
    // Base class members  
};  
class Derived1 : public Base {  
    // Derived1 class members  
};  
class Derived2 : public Derived1 {  
    // Derived2 class members  
};
```

## Hierarchical Inheritance

- In hierarchical inheritance, multiple derived classes inherit from a single base class.

```
class Base {  
    // Base class members  
};  
class Derived1 : public Base {  
    // Derived1 class members  
};  
class Derived2 : public Base {  
    // Derived2 class members  
};
```

## Hybrid Inheritance

- Hybrid inheritance is a combination of two or more types of inheritance.

```
class Base {  
    // Base class members  
};  
class Derived1 : public Base {  
    // Derived1 class members  
};  
class Derived2 : public Base {  
    // Derived2 class members  
};  
class Derived3 : public Derived1, public Derived2 {
```

```
// Derived3 class members  
};
```

## Constructors in Derived Class

- When a derived class object is created, the constructor of the base class is called first, followed by the constructor of the derived class.
- This ensures that the base class is properly initialized before the derived class adds its own initialization.

## Constructor Execution Order

- The order of constructor execution in inheritance is as follows:
  - The constructor of the base class is executed first.
  - The constructor of the derived class is executed next.
  - If there are multiple levels of inheritance, the constructors are called in the order of inheritance hierarchy, starting from the base class and moving down to the most derived class.

## Destructors in Derived Class

- When a derived class object is destroyed, the destructor of the derived class is called first, followed by the destructor of the base class.
- This ensures that the derived class can clean up its resources before the base class does.

## Destructor Execution Order

- The order of destructor execution in inheritance is as follows:
  - The destructor of the derived class is executed first.
  - The destructor of the base class is executed next.
  - If there are multiple levels of inheritance, the destructors are called in the reverse order of constructor execution, starting from the most derived class and moving up to the base class.

## Tomorrow's Topics:

- Constructor Execution with Parameters
- Function Overriding
- Overloading vs Overriding
- Polymorphism
- Types of Polymorphism
- Compile-Time Polymorphism
- Run-Time Polymorphism
- Virtual Functions
- Early Binding vs Late Binding
- Upcasting
- Downcasting
- dynamic\_cast
- Object Slicing
- Virtual Destructor
- Diamond Problem

- Virtual Base Class
- vtable & vptr Concept
- Pure Virtual Functions
- Abstract Class
- Interfaces
- Interface-style Thinking